
iss26

Release 1

Antonio Natali

Mar 06, 2026

CONTENTS:

1	Introduzione al corso	1
1.1	Occorre superare l'idea che il software sia solo codice	1
1.2	La pratica senza teoria	1
1.3	Cosa fare?	1
1.4	Il ruolo del testo di riferimento	2
1.5	Organizzazione del corso	3
1.6	Organizzazione delle attività	3
1.7	Modalità di (auto)valutazione	3
2	Primi passi operativi	5
2.1	Installare su PC	5
2.2	Inviare al docente	5
2.3	Il ruolo di Gradle	5
3	GradleIntro25	7
3.1	Concetti Fondamentali	7
3.2	Comandi Principali	7
3.3	Best Practices	8
3.4	Conclusione	8
4	La (auto)valutazione	9
4.1	Criteri di (auto)valutazione	9
4.1.1	Dettagli sul colloquio orale	9
5	TemaFinale24	11
5.1	The structural part of the building	11
5.2	The behavior of the Incinerator	12
5.3	The behavior of the OpRobot	12
5.4	The management of Containers	12
5.5	The ServiceStatusGUI	12
5.6	The behavior of the MonitoringDevice	13
5.7	Service users story	13
6	TemaFinale25	15
6.1	TF2025 Requirements	15
7	Project conway26Java	17
7.1	Il file README.md	17
7.2	conway26Java-build.gradle	21
8	conway26Java Dai requisiti al deployment	23

8.1	conway26Java-Requirements	23
8.2	conway26Java Analisi del problema	23
8.2.1	conway26Java-ArchitetturaLogica	24
8.2.2	conway26Java Piani di Testing funzionale	24
8.3	conway26Java-Costruttori	26
8.4	conway26Java-Progettazione&realizzazione	27
8.4.1	MainConwayLifeJava	27
8.4.2	conway26Java-Coding	28
8.4.3	Uno schema per partire	28
8.4.4	conway26Java-TestReports	28
8.4.5	conway26Java-JavaCodeCoverage	28
8.4.6	conway26Java-Esecuzione	29
8.5	conway26Java-Deployment	29
8.5.1	Generazione di un file jar eseguibile	29
8.6	Attivare GitHub Pages	30

INTRODUZIONE AL CORSO

Tenendo conto di quanto riportato in [Preludio](#), la moderna professione del costruttore di software implica che:

- occorre elevare il livello da ‘programmatore’ a Ingegnere del software di sistema, che si concentra maggiormente sulla validazione, sulla logica di business e sull’integrazione dei sistemi, piuttosto che sulla scrittura diretta di codice.
- L’IA non sarà più un semplice correttore ortografico, ma un collaboratore che suggerisce pattern di design e genera test

1.1 Occorre superare l’idea che il software sia solo codice

Il ruolo del costruttore umano di software rimane però centrale, richiedendo una profonda comprensione dei principi di progettazione, delle [architetture software](#) e delle [metodologie di sviluppo](#) oltre a una capacità di adattamento continuo alle nuove tecnologie e ai nuovi [paradigmi emergenti](#)

Infatti, il settore del software non è solo un campo tecnico, ma una **stratificazione di astrazioni**.

1.2 La pratica senza teoria ...

- “La pratica senza una base teorica è come un marinaio che sale su una nave senza timone e bussola, e che non ha mai la certezza di dove stia andando.” (*Leonardo da Vinci*)
- “La pratica senza una base teorica è cieca, così come la teoria senza la pratica è un mero esercizio intellettuale.”
- “La pratica senza una base teorica è puro tentativo ed errore (trial and error) destinato a non scalare.”

L’IA è bravissima nella “pratica” (generare codice che gira), ma non possiede la “teoria” (non capisce il perché profondo di una scelta architeturale). Quando sorge un problema, solo chi possiede la teoria sa dove mettere le mani.

1.3 Cosa fare?

In questo quadro, il corso prevede una stretta interazione tra la parte teorica e la parte pratica di laboratorio e lo **sviluppo cooperativo** (docente.studente e studente-studente) di casi di studio concreti.

- L’attività di laboratorio è volta a sviluppare abilità teorico/pratiche nell’analisi, progettazione e costruzione di sistemi software.

Come ogni attività umana, anche l’attività di costruzione del software si svolge in un contesto di riferimento, sia di tipo culturale, sia relativo agli obiettivi che si intendono raggiungere.

- Per meglio definire questo contesto (in continua evoluzione nel campo del software) è disponibile un testo, scritto con l’obiettivo di introdurre un percorso incrementale (teorico/pratico) di transizione dalla ‘tradizional’ programmazione ad oggetti alla costruzione di sistemi basati su componenti software distribuiti che comunicano mediante protocolli di rete.

Lo scenario applicativo di riferimento è quello dei [Sistemi a Microservizi](#) (di cui parleremo) e dei [Sistemi Embedded/IoT](#).

- L'intento è di dimostrare come l'ingegnere del software possa affrontare la costruzione di **sistemi eterogenei**, in domini applicativi diversi, avendosi di un insieme di concetti volti a superare [L'Abstraction gap](#) tra gli obiettivi di alto livello che si intendono raggiungere e i dettagli di basso livello legati alle tecnologie software e hardware disponibili.

Si vuole dimostrare come sia possibile costruire non solo prodotti finali, ma anche opportuni insiemi 'custom' di strumenti di supporto alla progettazione e allo sviluppo, avvalendosi di tecniche tipiche dei [Domain-Specific Languages](#).

1.4 Il ruolo del testo di riferimento

Il Testo di riferimento :

1. conviene sia aperto con il **browser Firefox**
2. contiene informazioni che ogni costruttore di software dovrebbe avere nel proprio bagaglio culturale

Leggere il testo

Il testo dovrebbe essere letto in modo sequenziale almeno fino al [Capitolo6](#):

1. [PRELUDIO](#) Forse c'è ancora lavoro ...
2. [COSA SI INTENDE PER SISTEMA](#) tanti tipi diversi, ma a noi interessano solo sistemi basati sul concetto di 'calcolo' ...
3. [COSA VUOL DIRE COMPUTARE](#) il concetto di 'computazione' ha subito una notevole evoluzione, ma in ogni caso a noi occorrono solo 'linguaggi di programmazione' ...
4. [I LINGUAGGI](#) sono fondati su modelli formali di computazione e fungono sia come ali sia come catene ... ed evolvono in continuazione
5. [SISTEMI SOFTWARE](#) possono appartenere ai semplici, complicati e anche ai complessi (ma, si spera, mai caotici)
6. [GAME OF LIFE DI CONWAY](#)

3. introduce un **percorso incrementale di transizione** dalla programmazione ad oggetti alla costruzione di sistemi software distribuiti e di [Sistemi a Microservizi](#), con una visione prospettica sui [Sistemi software complessi](#) (si consiglia la visione del video [Il Sorprendente Segreto della Sincronizzazione](#))

Consultare il testo

Dal [Capitolo6](#) in avanti, il Testo riporta contenuti specificatamente orientati al [CorsoISS2026](#)

La parte più orientata al corso:

- introduce librerie che realizzano l'idea astratta di [Interconnessione](#) (di cui parleremo) tra componenti software distribuiti che comunicano mediante protocolli di rete quali TCP, MQTT, CoAP, etc.
- introduce all'uso del paradigma computazionale ad attori [paradigma computazionale ad attori](#)

1.5 Organizzazione del corso

1. Parte A: *Dai programmi ai Sistemi a Microservizi*
2. Parte B: *Sviluppo di applicazioni distribuite* e sperimentazione di Sistemi IOT
3. Parte C: *Dai requisiti al prodotto* mediante Processi di sviluppo agili e incrementali basati su modelli

Nelle tre fasi, faremo riferimento a un *Filo conduttore* costituito dal *Game of Life* di Conway che implica

- un sistema semplice da capire e da costruire nel concentrato
- un sistema complicato nel distribuito
- un sistema complesso dal punto di vista dei risultati che presenta

comportamenti caotici

Per un esempio di comportamento caotico si veda: *Mappa Logistica*.

1.6 Organizzazione delle attività

1. *Primi passi operativi*
2. *Il nostro metodo di lavoro*

1.7 Modalità di (auto)valutazione

Si veda anche *La (auto)valutazione*

PRIMI PASSI OPERATIVI

2.1 Installare su PC

1. Installare **firefox** per la consultazione del materiale didattico
2. Installare *Java JDK 17* (*Windows->Preferences->Java->InstalledJRE*)
3. Installare *GIT* e un Git repository **PERSONALE PRIVATO** e **dare accesso al docente**
4. Installare *Gradle* (8.4, 8.6) (per accedere a *System Properties Advanced* su Windows: `sysdm.cpl`)
5. Installare *Eclipse IDE: Eclipse IDE for Java and DSL Developers*
6. Installare *Docker* e provarne l'uso

In un secondo momento:

1. Installare *Python (3.9.13)* (usando **Anaconda**)
2. Installare *IntelliJ*
3. Installare *Node.js*

2.2 Inviare al docente

- Compilare il file `templateToFill.html` nella sezione finale, relativa ai dati dello studente
- Stampare il file in **formato pdf** con nome `<CognomeNomeMatricola>` e inviare il file al docente (*antonio.natali@unibo.it*)

2.3 Il ruolo di Gradle

La costruzione di un sistema software non può oggi prescindere dall'uso di un **IDE** e di strumenti di come *Gradle*, uno strumento open source per automatizzare la costruzione (build) del software.

Gradle è ormai uno standard di fatto in questo settore ed è interessante non solo in quanto strumento, ma anche perchè applica i principi del *Domain Driven Design (DDD)* per modellare il suo proprio domain-building software.

Ne consegue che non poco tempo deve essere dedicato allo studio di questi tools e alla preparazione del file `build.gradle` che governa lo sviluppo e il deployment del sistema software.

Le rete offre numerosi tutorial su *Gradle*, come ad esempio: <https://tomgregory.com/gradle/gradle-tutorial-for-complete-beginners/> (*GradleIntro25*)

i Note

Per usare Eclipse insieme a Gradle, potrebbe essere necessario eseguire i seguenti passi (Grazie a *Stefano Arasi*):

1. Help->Eclipse MarketPlace
2. Search “Gradle”
3. Go to “BuildShop Gradle Integration 3.0” (the elephant) which is indicated as installed and clic on the grayed installed button.
4. Either click on update or uninstall and reinstall it, this will restart eclipse.

GRADLEINTRO25

Gradle è un potente strumento di build che consente l'automazione della compilazione, test, gestione delle dipendenze e distribuzione di software. Questo documento fornisce un riepilogo dei concetti fondamentali e dei comandi principali che un software designer deve conoscere per utilizzarlo efficacemente.

3.1 Concetti Fondamentali

1. **Build Script:** Il file *build.gradle* definisce le dipendenze e le configurazioni del progetto.
2. **Task:** Un'unità di lavoro in Gradle (compilazione, test, packaging, ecc.).
3. **Project:** Un'istanza di un'applicazione Gradle, che può contenere uno o più moduli.
4. **Dependency Management:** Gradle utilizza un modello dichiarativo per gestire le dipendenze.
5. **Plugins:** Espandono le funzionalità di Gradle (es. *java*, *application*, *spring-boot*).
6. **Gradle Wrapper:** (*gradlew*) permette di eseguire Gradle senza doverlo installare manualmente.

3.2 Comandi Principali

1. Inizializzazione e Informazioni sul Progetto

- Creare un nuovo progetto:

```
gradle init
```

- Visualizzare i task disponibili:

```
gradle tasks
```

- Mostrare le dipendenze del progetto:

```
gradle dependencies
```

2. Compilazione e Esecuzione

- Compilare il codice:

```
gradle build
```

- Eseguire un'applicazione Java:

```
gradle run
```

- **Pulire i file di build generati:**

```
gradle clean
```

3. Test e Qualità del Codice

- **Eseguire i test:**

```
gradle test
```

- **Generare un report della qualità del codice:**

```
gradle check
```

4. Gestione delle Dipendenze

- **Aggiornare le dipendenze:**

```
gradle dependencies --refresh-dependencies
```

- **Forzare il download di tutte le dipendenze:**

```
gradle build --refresh-dependencies
```

5. Gradle Wrapper

- **Generare lo script Wrapper:**

```
gradle wrapper
```

- **Utilizzare Gradle Wrapper invece di Gradle direttamente:**

```
./gradlew build
```

3.3 Best Practices

- **Utilizzare il Wrapper** per garantire la compatibilità del progetto tra gli sviluppatori.
- **Organizzare il build script** separando i plugin, le dipendenze e la configurazione dei task.
- **Evitare di hardcodare versioni delle dipendenze** e utilizzare *dependencyManagement* per una gestione centralizzata.
- **Abilitare build caching** per migliorare le performance della compilazione.

3.4 Conclusione

Gradle è uno strumento potente per la gestione dei progetti software. Comprendere i concetti chiave e i comandi principali aiuta un software designer a ottimizzare il processo di sviluppo e distribuzione delle applicazioni.

LA (AUTO)VALUTAZIONE

La valutazione finale è il punto terminale di un processo continuo di auto-valutazione che ogni costruttore di software fa durante e al termine della produzione di una applicazione, anche come possibile estensione di una [SCRUM Sprint retrospective](#).

4.1 Criteri di (auto)valutazione

1. La costruzione di un prototipo di sistema software che soddisfa l'insieme di requisiti (ad esempio [TemaFinale25](#)) mira a verificare il raggiungimento di adeguate abilità teorico/pratiche e può essere effettuata in modo individuale oppure in un gruppo di non più di tre studenti.
2. La discussione individuale orale che segue la demo del prototipo è volta a valutare il livello di conoscenza, consapevolezza e di autonomia analitico/progettuale del singolo studente.

4.1.1 Dettagli sul colloquio orale

- 48 h prima del colloquio, il codice del sistema finale deve essere stato pubblicato sul sito del prototipo (individuale o di gruppo), dandone relativa informazione via mail al docente.
- Il giorno del colloquio, ogni gruppo deve avere effettuato gli opportuni preparativi per la/le demo, in modo da essere subito operativo.

L'ordine di presentazione dei gruppi verrà opportunamente stabilito dal docente e si svolgerà come qui di seguito indicato.

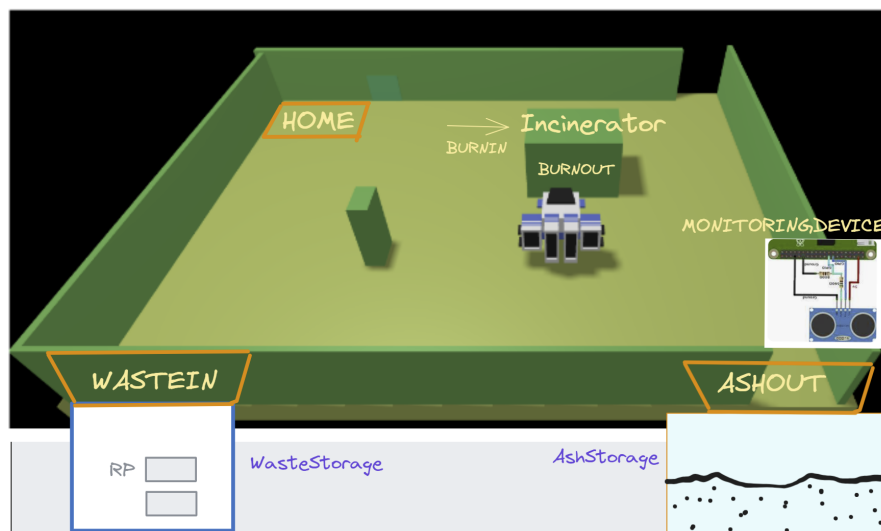
1. Presentazione (collettiva di gruppo) di una demo 'live' del sistema finale di durata 10-15(max) minuti. La demo deve mostrare la esecuzione di almeno un Test(Plan) automatizzato ritenuto significativo. Per applicazioni che NON usano robot reali NON sono ammessi video, che potrebbero essere invece utili per mostrare il funzionamento di robot reali o di sistemi che includono il RaspberryPi o altri dispositivi
2. Presentazione (collettiva di gruppo) del progetto del sistema e della sua relazione con la fase di analisi. In questa fase è RICHiesta la preparazione di 5-7 SLIDES di illustrazione del processo e dell'architettura finale con figure e (se ritenuto utile) riferimenti al codice. Al termine di queste fasi il gruppo può raggiungere un punteggio massimo di 27/30.
3. Domande (per esempi, si veda qui sotto) rivolte dal docente a singole persone, riguardo al prodotto, al progetto e alla analisi del problema/requisiti. Al termine di questa fase, il **singolo studente** può raggiungere un punteggio massimo di 29/30.
4. Altre domande rivolte dal docente a singole persone. Al termine di questa fase, il **singolo studente** può raggiungere il punteggio di 30 e lode.

TEMAFINALE24

A company intends to build a WasteIncineratorService to treat waste by burning it and requires a software system service (WIS) that controls a robot (called OpRobot) in order to move the waste.

5.1 The structural part of the building

The Incinerator is situated within a service area (rectangular, flat) as shown in the following picture:



Outside the service area, there are:

1. a WasteStorage container, devoted to store waste material in the form of Roll Packets (RP). Each RP has a weight $WRP = 50 \text{ Kg}$ (approximately);
2. a AshStorage container, devoted to store the ashes produced by the incineration process. This container can store (approximately) the ashes of 3-4 RP.
3. a MonitoringDevice, composed by a a Sonar and a Led working on a RaspberryPi.

The waste can be introduced into the Incinerator through its BURNIN port, while the ash produced by the Incinerator can be extracted using the BURNOUT port.

The service area includes:

1. a WASTEIN port, that can be used to enter into the service area the RP of waste.
2. a ASHOUT port, that can be used to move out of the service area the ash produced by the incineration process.

5.2 The behavior of the Incinerator

- The Incinerator is able to perceive a proper activation command sent by using a wireless (*wifi*, *bluetooth*) connection.
- The Incinerator can process one RP at the time. The burning process requires (approximately) BTIME seconds.
- At the end of a burning phase, the Incinerator emits a (acustic, or other) signal that can be perceived by the OpRobot and by the WIS.

5.3 The behavior of the OpRobot

The company provides a DDR robot (and its own control software), that should be used as the physical actuator for the behavior of the OpRobot, that can be listed as follows:

1. stay in the HOME location when there is no work to do;
2. if the WasteStorage container is not empty, the AshStorage container is not full, and the Incinerator is not in a burning phase, move to the WASTEIN port. If one of the conditions is **not true**, wait at HOME, until it becomes *true*.
3. get a RP from the WasteStorage container;
4. move to the BURNIN port and deposit the RP into the Incinerator;
5. move to the HOME location when the Incinerator is in its burning phase;
6. move to the BURNOUT port to extract the ash, when the Incinerator has completed a burning phase;
7. move to the ASHOUT port and deposit the ash into the AshStorage container;
8. reconsider the point 2 (and go back to HOME, if it is the case).

5.4 The management of Containers

- A new RP is put into the WasteStorage container by some external agent.
- The WasteStorage owns a *weighing device* (Scale) that reports the current weight of all the RP currently stored into the container. The container can be considered empty when the value of the Scale is (approximately) 0.
- Another external agent provides to remove the ash from the AshStorage container. This action modifies the value measured by the Sonar of the *MonitoringDevice*.

The WIS can acquire information from the input devices Scale and Sonar through software supports that must be properly designed and implemented.

5.5 The ServiceStatusGUI

The WIS system must also provide a ServiceStatusGUI (SSGUI) that must show:

1. the current state of the WasteStorage, i.e. the number of RP currently stored in it;
2. the current state of the AshStorage, i.e. an indication of the level of its capacity currently used;
3. the current state of the Incinerator, i.e. if it is burning or not;
4. the state of the OpRobot, i.e. an indication of its current location in the service area and of the job that it is doing.

5.6 The behavior of the MonitoringDevice

The Sonar of the MonitoringDevice is used to measure the level of the ash in the AshStorage container, by measuring the distance between the top of the ash and the *Sonar* itself. When the distance is less than a prefixed value **DLIMIT**, the AshStorage container is considered full.

The Led is used as a warning device, according to the following rules:

- the Led is **on** when the Incinerator is burning a RP.
- the Led is **off** when the Incinerator is not burning.
- the Led **blinks** while the AshStorage is full or the AshStorage is empty

5.7 Service users story

As an user of the WIS, I see that:

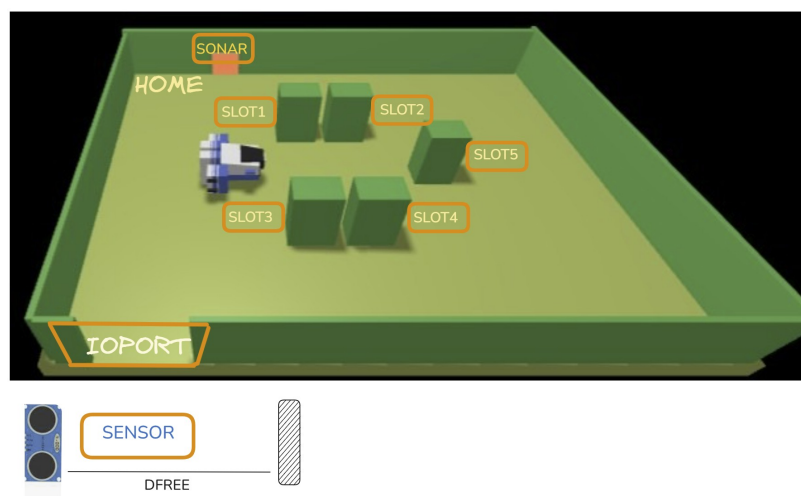
1. The OpRobot is somewhere in the service area (initially in the HOME location).
2. If the WasteStorage container is not empty, and AshStorage container is not full, (i.e. the *Led* is not blinking) and the Incinerator is not burning, the OpRobot moves to the WASTEIN port and gets a RP from the WasteStorage container. Otherwise, it returns to HOME, if not already here.
3. From now on, *The ServiceStatusGUI* shall properly change (part of) its content.
4. The OpRobot moves to the BURNIN port and deposits the RP into the Incinerator (the Led is on).
5. While the Incinerator is burning, the OpRobot moves to its HOME port.
6. When the Incinerator has completed a burning phase, the Led is off, and the OpRobot moves to the BURNOUT port, picks up the ash and moves to the ASHOUT port.
7. The OpRobot deposits the ash into the AshStorage container and the Sonar gives a value less than the previous one.
8. The OpRobot restarts from point 1.

TEMAFINALE25

A *Maritime Cargo shipping company* (from now on, simply company) intends to automate the operations of load of freight in the ship's cargo hold (or simply hold). To this end, the company plans to employ a *Differential Drive Robot* (from now, called cargorobot) for the loading of goods (named products) in the ship's hold.

The products to be loaded must be placed in a container of predefined dimensions and registered, by specifying its weight, within a database, by using a proper service (productservice). After the registration, the *productservice* returns a **unique product identifier** as a natural number PID, $PID > 0$.

The hold is a rectangular, flat area with an Input/Output port (IOPort). The area provides 4 slots for the product containers.



In the picture above:

- The slots depict the *hold storage areas*, when they are occupied by *product containers*
- The slots5 area is permanently occupied, while the other slots are initially empty
- The sensor put in front of the IOPort is a sonar used to detect the presence of a product container, when it measures a distance D , such that $D < DFREE/2$, during a reasonable time (e.g. 3 secs).

6.1 TF2025 Requirements

The company asks us to build a software systems (named cargospace) that:

1. is able to receive the **request to load** on the cargo a product container already registered in the productservice.

The request is rejected when:

- the product-weight is evaluated too high, since the ship can carry a maximum load of $\text{MaxLoad} > 0$ kg.
- the hold is already full, i.e. the 4 slots are already occupied.

If the request is accepted, the cargoservice associates a slot to the product PID and returns the name of the reserved slot. Afterwards, it waits that the product container is delivered to the ioport. In the meantime, other requests are not elaborated.

2. is able to detect (by means of the sonar sensor) the presence of the product container at the ioport
3. is able to ensure that the product container is placed by the cargorobot within its reserved slot. At the end of the work:
 - the cargorobot should return to its HOME location.
 - the cargoservice can process another *load-request*
4. is able to show the current state of the hold, by means of a dynamically updated web-gui.
5. interrupts any activity and turns on a led if the sonar sensor measures a distance $D > D_{\text{FREE}}$ for at least 3 secs (perhaps a sonar failure). The service continues its activities as soon as the sonar measures a distance $D \leq D_{\text{FREE}}$.

PROJECT CONWAY26JAVA

Costruiamo?. Si veda [Processi di costruzione del software](#) .

Seguendo quanto detto in [Il nostro metodo di lavoro](#), consideriamo questo progetto come lo *Sprint1* per la costruzione incrementale ed evolutiva di un prodotto software.

Si veda [GAME OF LIFE DI CONWAY](#)

Premessa

Si suppone di avere definito un sito GIT che contiene solo *Il file README.md* e di avere clonato questo sito entro una directory di lavoro (es. **iss2026**) sul nostro PC.

7.1 Il file README.md

Il file **README.md** di un sito GIT è molto più di un semplice file di testo: è il biglietto da visita e lo strumento di marketing delle nostre attività.

Inoltre potrebbe fornire indicazioni utili come manuale d'uso delle applicazioni.

Il file deve essere scritto usando il [Linguaggio Markdown](#).

```
# issLab2026
Laboratorio di <b>Ingegneria dei Sistemi Software</b> a.a. 2025/2026 di Cognome Nome_
↪Matricola

[Testo di riferimento:](https://anatali.github.io/issLab2026/_static/docs/Protobook.pdf)
<!-- comment: [ancora personalizzata] -->

<h2 id="ParteA">Parte A: Dai programmi ai Sistemi a Microservizi</h2>

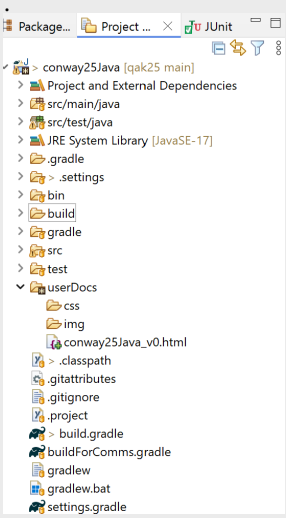
### Sistema ConwayLife in locale
[Riferimento: conway26Java Dai requisiti al deployment](https://anatali.github.io/
↪issLab2026/Project%20conway26Java.html#conway26java-dai-requisiti-al-deployment)
* [ConwayLife Sprint1](ConwayLife/Sprint1/conway26Java): impostazione di un primo_
↪prototipo
  in Java con dispositivi Mock di I/O. <i>Distribuzione</i>: file jar.
* [ConwayLife Sprint2](ConwayLife/Sprint2/conway26Java): <b>evoluzione</b> del primo_
↪prototipo
  con un dispositivo di output realizzato in Swing. <i>Distribuzione</i>: file jar
### Sistemi come servizi
* ...
### Sistema ConwayLife con pagine HTML
```

(continues on next page)

(continued from previous page)

* [ConwayLife Sprint3(ConwayLife/Sprint3/conway26Java): evoluzione del sistema in locale usando una pagina HTML come dispositivo di I/O. <i>Distribuzione</i>: Docker yaml.

Creiamo il progetto e impostiamo in Eclipse



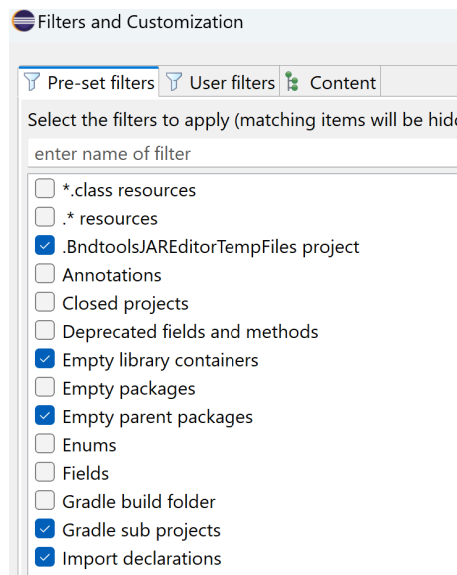
Inizializzazione

- operiamo nella directory di lavoro **iss2026**
- copiamo in questa la directory **unibolibs** (dal sito del corso)
- creiamo la subdirectory **iss2026/conway26Java**
- nella directory **iss2026/conway26Java** eseguire: `gradle init 1 2 - -`
- importiamo il progetto in un workspace eclipse
- definiamo il [build.gradle](#)
- aggiungiamo *Java nature*
- creiamo il source folder `src`
- eseguiamo da un terminale `gradlew eclipse` per generare i file per Eclipse
- scriviamo un file **HelloWorld.java** di prova

Esecuzione del prodotto

- `gradlew run` o da Eclipse

Settaggio filtri per rendere visibili **resources** e **Gradle build folder**



Parte applicativa: la fase di analisi e progettazione

- leggiamo [Il gioco come caso di studio](#)
- aggiungiamo folder `userDocs` e al suo interno:
 - il file di nome `conway26Java_v0.html` ricavato da [template](#) che ha il ruolo di un *diario di bordo* (con varie versioni)

- la subdirectory `userDocs/css` per gli stili usati dal diario
- la subdirectory `userDocs/img` per le immagini usate dal diario
- riportiamo nel file `conway26Java_v0.html` i dati del progettista e scriviamo le sezioni **Requirement analysis** e **Problem analysis** con le opportune motivazioni
- pensiamo alle procedure di testing funzionale (si veda [procedure di testing funzionale](#)) e scriviamo la sezione **Test plans**
- impostiamo il progetto e scriviamo la sezione **Project** (prima di scrivere il codice)

Parte applicativa: la fase di realizzazione

- aggiungiamo (se non esiste) come *sourcefolder*: `src/main/java` e le altre directory specificate in [build.gradle](#)
- aggiungiamo il package `conway`
- procediamo con la produzione del codice
- aggiungiamo procedure di testing

7.2 conway26Java-build.gradle

```

plugins {
    id 'application'
    id 'java'
    id 'eclipse'
}
version '1.0'
java {
    toolchain.languageVersion.set(JavaLanguageVersion.
↳ of(17))
}
repositories {
    mavenCentral()
    flatDir { dirs '../unibolibs' }
}
dependencies {
    /* UNIBO ***** */
    implementation name: 'uniboInterfaces'
    implementation name: '2p301'
    implementation name: 'unibo.basicomm23-1.0'
}
sourceSets {
    main.java.srcDirs += 'src'
    main.java.srcDirs += 'src/main/java'
    main.java.srcDirs += 'src/main/kotlin'
    main.java.srcDirs += 'src/main/resources'
    test.java.srcDirs += 'src/main/java/test' //
↳ test is specific
    test.java.srcDirs += 'src/main/kotlin/test'
}
eclipse {
    classpath {
        sourceSets -= [sourceSets.main, sourceSets.test]
    }
}
application {
    //Define the main class for the application.
    mainClassName = 'conway26Java.HelloWorld'
}
jar {
    println("building jar")
    from sourceSets.main.allSource
    from('.') {
        include '*.pl'
        include '*.json'
    }
    manifest {
        attributes 'Main-Class': "$mainClassName"
    }
}
task dovesiamo {
    println("projectDir= $projectDir")
    println("buildDir = $buildDir")
}

```

In questa versione vi sono dipendenze solo relative alla libreria JUnit per fare testing.

unibolibs contiene le librerie

- 2p301.jar
- uniboInterfaces.jar
- unibo.basicomm23-1.0.jar.

CONWAY26JAVA DAI REQUISITI AL DEPLOYMENT

Il nostro metodo di lavoro impone che si debba partire dai requisiti, che il committente ha specificato come segue:

8.1 conway26Java-Requirements

1. Realizzare una versione in Java del gioco **Life** di Conway, come gioco zero-player. Il gioco consiste nell'introdurre una **Griglia di Celle** il cui stato (cella 'viva' o cella 'morta') evolve come stabilito dalle [regole di ConwayLife](#)
2. L'utente umano deve poter:
 - specificare la configurazione iniziale della griglia del gioco
 - vedere l'evoluzione del gioco in forma opportuna (si veda [Problema della vista del gioco](#))
 - fermare e far ripartire l'evoluzione del gioco
 - pulire (a gioco fermo) la configurazione della griglia del gioco

Il passo successivo è impostare l'analisi del problema e delineare una architettura logica del sistema.

8.2 conway26Java Analisi del problema

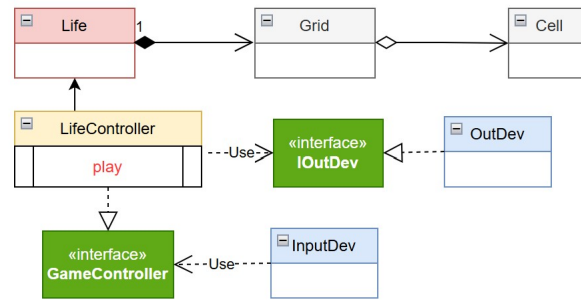
Una prima analisi del problema si può trovare in [Analisi del problema GofLife](#), mentre un primo schema organizzativo del sistema da realizzare si trova in [Progetto ConwayLife26](#)

Lo schema rappresenta un [modello del sistema](#), espresso utilizzando notazioni grafiche in [stile UML](#). Lo riportiamo qui sotto, come **prodotto della analisi del problema**, prima ancora che di progetto.

Infatti:

1. I requisiti menzionano una *Griglia di Celle*. Questi termini sono formalizzati nelle classi **Grid** e **Cell** ritenendo adeguato che questi enti siano realizzati in software come [POJO](#).
2. Le [regole di Conway](#) sono il 'core business' applicativo e fanno riferimento a Grid. La specifica di queste regole è affidata a un altro POJO, istanza di una classe **Life**.
3. Il gioco è zero-player, ma deve essere controllato dall'utente attraverso dispositivi di I/O. Come analisti, non riteniamo che questo compito sia responsabilità di nessuna delle classi precedenti, per cui si ritiene necessario introdurre un nuovo POJO, di classe **LifeController**
4. I dispositivi di I/O potrebbero essere i più diversi. Occorre rendere *LifeController* indipendente dai dettagli tecnologici dell'I/O definendo opportuni **contratti** (formalizzati come **interface** Java) che i dispositivi dovrenno rispettare.

8.2.1 conway26Java-ArchitetturaLogica



Questo modello è stato definito in fase di analisi del problema per delineare una **architettura (logica)** del sistema che:

- tiene conto dei **Principi SOLID** separando le responsabilità dei diversi componenti (espressi in **stile OOP**)
- imposta il componente **LifeController** in modo tale che la *parte proattiva* del gioco sia indipendente dalle parti che realizzano l'input-output di interazione con l'utente umano e dai dettagli tecnologici di realizzazione dei dispositivi di I/O.
- imposta il sistema software in modo molto diverso (e **migliore**) da come si presenta nel codice (ispirato al **Paradigma Funzionale**) **ConwayLife.py** proposto dalla AI, come detto nel **preludio**

i Perché migliore?

- esprime in modo esplicito (con opportune classi Java) i concetti-base del dominio applicativo (**Grid** e **Cell**) stabilito nel *testo dei requisiti*
- l'entità **Cell** non viene 'ridotta' a un array di interi o booleani
- introduce una entità (**Life**) che ha come sola responsabilità le regole del gioco, separandola da una diversa entità (**LifeController**) responsabile della interazione con l'utente (che nella versione Python viene espressa nel *main* !!)
- prevede la possibilità di **configurare** il sistema con diversi dispositivi di I/O, lasciando inalterato il codice relativo al controllo del gioco, come anticipato in **MainConwayLifeJava.java**
- ammette la possibilità che **LifeController** possa interrompere e riavviare la esecuzione del gioco, cosa che la versione Python non fa

- permette di definire un primo insieme di **test funzionali** concentrati sulla **classe Life**, che definisce il 'motore logico' del gioco.

8.2.2 conway26Java Piani di Testing funzionale

La introduzione di un Piano di test funzionale fin dalle fasi di analisi, è fondamentale per guidare la fase di realizzazione del codice. Ma sorge un problema:

come si fa a definire un piano di test funzionale se non si è ancora scritto una riga di codice?

La risposta consiste nel basarsi sul cosa (**WHAT**) il sistema dovrà fare e non sul come (**HOW**) verrà realizzato il sistema software.

Prendiamo ad esempio la classe **Life** che definisce il motore logico del gioco. Dalla specifica del gioco, possiamo dedurre che questa classe dovrà sostenere almeno a un insieme di funzionalità esprimibile in modo formale mediante una **sinterface** Java:

```

public interface LifeInterface {
    /** Restituisce il numero di righe e colonne della griglia */
    int getRows();
    int getCols();
    /** Imposta lo stato di una cella */
    void setCell(int x, int y, boolean alive);
    /** Restituisce lo stato di una cella specifica */
    boolean isAlive(int x, int y);
    /** Calcola l'evoluzione dello stato alla generazione successiva */
    void nextGeneration();
    /** Pulisce la griglia (nessuna cella viva) */
    void clear();
    /** Restituisce una rappresentazione grafica testuale della griglia */
    public String gridRep( );
}

```

A questo punto, possiamo definire un *piano di test funzionale* per il sistema, basandoci sui requisiti del gioco e sulla logica del gioco stesso. Ad esempio:

conway26Java-Impostazione Testing

```

public class ConwayLifeTest {

    @Before
    public void setup() {
        System.out.println("setup");
    }

    @After
    public void down() {
        System.out.println("down");
    }
}

```

conway26Java-testOscilla

```

@Test
public void testOscilla() {
    System.out.println("testOscilla -----" );
    LifeInterface liferules = new Life(5, 5);
    // Configurazione orizzontale
    liferules.setCell(2, 1, true);
    liferules.setCell(2, 2, true);
    liferules.setCell(2, 3, true);
    System.out.println("testOscilla | Stato Iniziale:\n" + liferules.gridRep());

    liferules.nextGeneration();
    System.out.println("testOscilla | after 1 gen:\n" + liferules.gridRep());
    // Verifica che sia diventato verticale
    assertTrue(liferules.isAlive(1, 2));
    assertTrue(liferules.isAlive(2, 2));
    assertTrue(liferules.isAlive(3, 2));
    assertFalse(liferules.isAlive(2, 1));
}

```

(continues on next page)

(continued from previous page)

```

liferules.nextGeneration();
System.out.println("testOscilla | after 2 gen :\n" + liferules.gridRep());
// Verifica che sia tornato orizzontale (Periodo 2)
assertTrue(liferules.isAlive(2, 1));
assertTrue(liferules.isAlive(2, 2));
assertTrue(liferules.isAlive(2, 3));
}

```

conway26Java-testOscillaFromFile

Un test potrebbe anche essere impostato in modo *Data-Driven*, leggendo la configurazione iniziale da un file.

```

@Test
public void testOscillaFromFile() throws Exception {
    // Carico un Blinker (periodo 2)
    boolean[][] initial = PatternLoader.loadFromResource("src/test/resources/blinker.txt", 5, 5);

    Life liferules = new Life(initial);

    System.out.println( liferules.formatGrid() );
    System.out.println( "-----" );

    liferules.nextGeneration(); // Generazione 1 (cambia stato)
    System.out.println( liferules.formatGrid() );
    System.out.println( "-----" );
    liferules.nextGeneration(); // Generazione 2 (deve tornare all'originale)
    System.out.println( liferules.formatGrid() );
    System.out.println( "-----" );

    assertEquals("L'oscillatore deve tornare allo stato iniziale dopo 2 passi",
        initial, liferules.getGrid());
}

```

8.3 conway26Java-Costruttori

Scrivere un *TestPlan* prima che la classe *Life* esista potrebbe indurre a decidere subito:

“Come passo la (specifica della) griglia? Con una matrice di booleani? Con un array di oggetti *Cell*? Come leggo il risultato?”.

Tuttavia ci sono modi per ‘differire’ queste scelte.

La procedura di testing *conway26Java-testOscillaFromFile* passa al costruttore di *Life* una griglia preconfigurata di nome *initial*. Se il costruttore di *Life* scrive *this.grid=initial*, si copia un riferimento. Ciò può essere pericoloso, perchè una modifica (dall’esterno di *Life*) di *initial* modifica anche l’interno della classe. Per evitare problemi, è opportuno fare una **deep copy** dell’array:

```

//Da Java8 in avanti
private boolean[][] deepCopyJava(boolean[][] original) {
    return Arrays.stream(original)
        .map(boolean[]::clone)

```

(continues on next page)

(continued from previous page)

```

        .toArray(boolean[][]::new);
    }

```

i Come Gemini spiega il codice

- `Arrays.stream(original)`: Prende la matrice (che è un “array di array”) e la trasforma in un flusso di dati (Stream). Ogni elemento di questo stream è una singola riga (`boolean[]`).
- `.map(boolean[]::clone)`: Per ogni riga che passa nello stream, viene chiamato il metodo `clone()`.
 - L’operatore `::` (*Method Reference*) dice a Java: “Prendi ogni riga e fanne una copia indipendente”.
 - Questo assicura che le righe della nuova matrice siano fisicamente diverse da quelle della vecchia.
- `.toArray(boolean[][]::new)`: Infine, raccoglie tutte queste nuove righe clonate e le impacchetta in una nuova matrice bidimensionale (`boolean[][]`).

8.4 conway26Java-Progettazione&realizzazione

La *conway26Java-ArchitetturaLogica*, *Life* definisce la griglia come un oggetto di classe `Grid` che contiene al suo interno una array (`Cell[][] gridrep`) di `Cell` che danno ‘**consistenza ontologica**’ alle celle, non più ‘ridotte’ ad array di `boolean` o di `integer`.

La procedura di testing *conway26Java-testOscilla* è più aderente a questa impostazione in quanto passa al costruttore di *Life* il numero di righe e colonne, lasciando alla classe la responsabilità (e l’opportunità) di definire la griglia come crede.

Si potrebbe anche pensare (con riferimento ai *design patterns*) di introdurre un *Factory method* in cui le dimensioni della griglia possono essere:

- stabilite di default
- lette da un file di configurazione
- passate come parametri al metodo

8.4.1 MainConwayLifeJava

Il Main (o **Composition Root**) è il luogo in cui la teoria dell’architettura incontra la pratica dell’esecuzione. In questa fase, invece di lasciare che le classi si istanzino tra loro (creando accoppiamento), è il programmatore a “iniettare” le dipendenze.

Ecco come Gemini propone di assemblare il sistema seguendo il diagramma UML:

```

public class Main {
    public static void main(String[] args) {
        // 1. Inizializziamo il Modello (Il "Cosa")
        // Life crea internamente la sua Grid (Composizione)
        Life gameModel = new Life(20, 40);

        // 2. Inizializziamo l'Output (Il "Dove")
        // Possiamo decidere qui se usare la Console o una GUI
        IOutDev outputDevice = new OutDev();

        // 3. Inizializziamo il Controller (Il "Come")
    }
}

```

(continues on next page)

(continued from previous page)

```

// Iniettiamo il modello e l'output nel controller (Dependency Injection)
GameController lifeController = new LifeController(gameModel, outputDevice);

// 4. Inizializziamo l'Input (L' "Innesco")
// L'input comunica con il controller
InputDev inputDevice = new InputDev(lifeController);

// 5. Avvio del sistema
System.out.println("Sistema S: Inizializzazione completata.");
inputDevice.onStartButtonPressed();
}
}

```

8.4.2 conway26Java-Coding

Il **passaggio dal modello al codice** richiede che quanto definito dall'analista sia ben compreso dal progettista e che questi lo traduca in codice in modo confacente alla specifica.

Anche in questa fase, la AI può aiutare: si veda conway26Java-Chiesto a Gemini. In pratica, utilizziamo la AI come un 'generatore di codice' **guidato dal nostro modello** di sistema e non semplicemente da quanto la AI trova in rete.

i Un prompt per Gemini

La figura allegata vuole definire l'architettura logica di un sistema software che realizza il gioco Life di Conway. Vorrei che tu ricavassi i concetti importanti che si possono desumere da questo daigramma (in stile UML)

— Uno schema per partire —

8.4.3 Uno schema per partire

Per agevolare questa transizione, è disponibile un workspace di riferimento nella cartella [ConwayLifeProject](#) (in [ConwayLifeSprint1](#)). Si noti che:

- sono definiti due package (`domain` e `devices`) per non mescolare il codice relativo al dominio e il codice relativo ai dispositivi di I/O.
- le interfacce `IOutDev.java` e `GameController.java` sono definite nel package **domain** per ribadire che sono i dispositivi che dovranno adattarsi alle esigenze del dominio e non viceversa

8.4.4 conway26Java-TestReports

Una volta definito il codice, le specifiche di testing possono essere eseguite come `Run As Junit Test` oppure attivando gradle:

- `gradlew test` e aprire il file `build\reports\tests\test\index.html` per vedere il risultato del test

8.4.5 conway26Java-JavaCodeCoverage

Aggiamo nel file `build.gradle` il **plugin JaCoCo** e la configurazione del task `jacocoTestReport`:

```

jacocoTestReport {
    dependsOn test // Il report viene generato solo dopo l'esecuzione dei test
}

```

(continues on next page)

(continued from previous page)

```
reports {
    xml.required = false
    csv.required = false
    html.outputLocation = layout.buildDirectory.dir('reports/jacoco')
}
}
```

Ora possiamo generare report che mostrano quali righe di codice sono state “visitate” dai test JUnit:

- `gradlew test jacocoTestReport` e aprire il file `build\reports\jacoco\index.html` per vedere il risultato del test

jacoco-sessions.html

Questo file è uno strumento di diagnostica che dice “quando” e “come” è avvenuta la raccolta dei dati. Serve a confermare che il “motore” di JaCoCo stia effettivamente analizzando i file `.class` giusti nel momento in cui JUnit esegue i test.

8.4.6 conway26Java-Esecuzione

- In `build.gradle` definiamo `mainClassName='conway26Java.MainConwayLifeJava'`
- `gradlew run` per eseguire il sistema

8.5 conway26Java-Deployment

Al momento, ci limitiamo a generare un file jar con le classi dello Sprint1:

Il file .jar

Un file `.jar` (**Java ARchive**) è un formato compresso basato su ZIP utilizzato per raggruppare classi Java, risorse (immagini, suoni) e metadati in un unico file per una facile distribuzione ed esecuzione. Richiede Java Runtime Environment (**JRE**) installato per funzionare e spesso contiene un file “manifest” per definire la classe principale eseguibile.

- `gradlew jar` per generare il file jar `conway26Java-1.0.jar` nella directory `build\libs`

Il file non è eseguibile, perchè non include le librerie per le dipendenze. Tuttavia esso può essere usato come libreria nello *Sprint2* del progetto.

8.5.1 Generazione di un file jar eseguibile

Volendo generare codice eseguibile autocontenuto, il modo più semplice è usare il task **distZip** (o *distTar*) reso disponibile dal plugin `application`.

Questo task impacchetta tutto il necessario per far girare il programma (file JAR, dipendenze esterne e script di avvio per Windows/Linux) in un unico archivio compresso. Il task deposita il risultato nella directory `build/distributions/conway26Java-1.0.zip`

Il file zip contiene due directory:

- **bin/** : Contiene gli script eseguibili (uno shell script per Linux/Mac e un file `.bat` per Windows). Questi script impostano automaticamente il `CLASSPATH` e lanciano la classe *Main*

- **lib/** : Contiene il file JAR del sistema più tutti i file JAR delle librerie esterne.

Per eseguire il sistema:

1. scompattare il file `build/distributions/-conway26Java1.0.zip` in una directory
2. nella directory `bin/`, eseguire (in Windows) lo script

```
conway26Java
```

Generare un file jar ‘fat’

In alternativa, si può creare un file jar “fat” che ingloba i file jar delle librerie esterne. Un esempio di un tale task è come ad esempio il task `jarfat(type: Jar) { ... }` definito in: <https://github.com/anatali/issLab2026/blob/main/ConwayLife/Sprint1/conway26Java/build.gradle>. Il comando:

```
gradlew jarfat
```

crea il file `build/libs/conway26JavaFat-1.0.jar` che può essere eseguito con il comando:

```
java -jar conway26JavaFat-1.0.jar
```

8.6 Attivare GitHub Pages

Per rendere leggibile il file `conway26Java_v0.html` direttamente dal proprio repository GIT, procediamo come segue:

1. Premessa: il sito deve essere **public**
2. Creiamo la cartella **docs** e poi:
 - copiamo in `docs` il file `conway26Java_v0.html` insieme alle directories di `userDocs` (se sono usate dal file) `css` e `img`
 - inseriamo un file **index.html**
3. Attiviamo **le GitHub Pages**:
 - Vai in *Settings* del repository GIT.
 - Clicca su *Pages* nella colonna a sinistra.
 - Sotto “Build and deployment”, imposta la sorgente su **“Deploy from a branch”**.
 - Scegli il branch (es. **main**) e la cartella **/docs**.
 - Clicca su Save.

Dopo un paio di minuti, il sito sarà online all’indirizzo `https://utente.github.io/nome-repo/`.